

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2024/2025

SEPTEMBER EXAMINATION

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

TUESDAY, 8 OCTOBER 2024

TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONOURS)

Instructions to Candidates:

This question paper consists of FIVE (5) questions .

Answer **ALL** questions in **Section A** and **ONLY ONE (1)** question in **Section B**. Each question carries 25 marks.

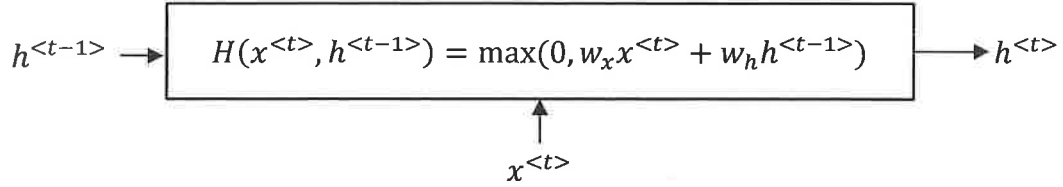
Should a candidate answers more than ONE (1) question in section B, marks will only be awarded for the **FIRST** question in that section in the order the candidate submits the answers.

Candidates are allowed to use a calculator.

Answer questions only in the answer booklet provided.

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**SECTION A (Answer all questions)**

Q1. The following figure shows a recurrent neural network (RNN) with ReLU activations.



- (a) Suppose that the RNN parameters $(w_x, w_h) = (0.8, 0.2)$. At time step t , the input $x^{<t>} = 2$ and previous hidden state $h^{<t-1>} = -3$ while the upstream gradient $\frac{dJ}{dh^{<t>}} = 0.5$.
- Draw the computational graph for $H(x^{<t>}, h^{<t-1>})$ for a single time step. Use only multiplication ($\times$), addition ($+$) and max ($\max$) as the basic computational units. (6 marks)
 - Perform forward propagation on the cell at time step t . (3 marks)
 - Perform backpropagation to compute the gradients $\frac{dJ}{dw_x}$, $\frac{dJ}{dw_h}$ and $\frac{dJ}{dh^{<t-1>}}$. (6 marks)
- (b) RNN may encounter some issues when training on sequence data.
- Explain why **vanishing gradient** is an issue when training a RNN network. (2 marks)
 - Will the gradient at w_x cause vanishing gradient issue? Justify your answer. (2 marks)
 - Why does the neuron fail to update its parameter when the weight is initialized to big random negative values? Explain how it happens using the computational graph in (a)(i). (3 marks)
 - Justify if the issue in (b)(iii) will persist if we replace ReLU with tanh activation. (3 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Q2. A log file stores daily measurements from various sensors in Kampar from year 1920 to 2023. These sensors record four key atmospheric variables (4 *features*), including temperature, humidity, wind speed and pressure on a daily basis. Your task is to build an LSTM network to predict the amount of rainfall (mm) given the sensor data from previous 20 days.

- (a) Answer the following questions on data preprocessing.
- (i) Suggest **ONE(1)** strategy for extracting samples from the raw data and **ONE(1)** strategy for splitting the dataset into training and test sets. (4 marks)
 - (ii) Some sensors failed to capture data on certain days. Suggest ways to overcome the issue. (2 marks)
 - (iii) Suggest **ONE(1)** data augmentation to enrich the training set for the sensor data. (2 marks)
 - (iv) What is the benefit of standardizing the data to zero mean? (2 marks)
- (b) The LSTM network is trained with batch size 256.
- (i) Explain why LSTM is appropriate for the task. (2 marks)
 - (ii) Which architecture (many-to-many, one-to-many, many-to-one, encoder-decoder) is appropriate for the task? (2 marks)
 - (iii) Assume a *batch first* data format. What is the shape of a batch data? (2 marks)
 - (iv) Design an LSTM network by providing the details and output shape of all **layers or operations** in Table 2.1. Then, specify the **cost function** to train the network.

Table 2.1: LSTM network for rainfall prediction

No	Layer / Operation*	Shape of output tensor
1	...	...
...	...	...

* Include critical information, e.g., LSTM: (*input_size*, *hidden_size*, *num_layers*, *bidirectional* and *batch_first*) and LINEAR: (*in_features* and *out_features*).

(7 marks)

- (v) After rigorous and successful evaluation, the model is published online to general public. Some users feedback that the model is not working well. Specify **ONE(1)** possible reason why this happens. (2 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

- Q3. (a) (i) Consider the following input volume and filter of a **conv2D** layer with stride 2 and padding 0. Compute the output volume.

		Input volume					Filter	
							W	b
							0 2	2
							2 0	
1 st channel:		1	2	1	0			
		2	0	2	5			
		1	0	3	0			
		2	2	-1	2			
		1	2	2	0		1 0	
2 nd channel:		0	2	1	3		0 1	
		1	1	2	1			
		-2	1	0	5			

(8 marks)

- (ii) Figure 3.1 shows a plain block. Add the skip connection to convert the plain block into a residual block. Then, explain how adding the skip connection helps to train very deep neural network.

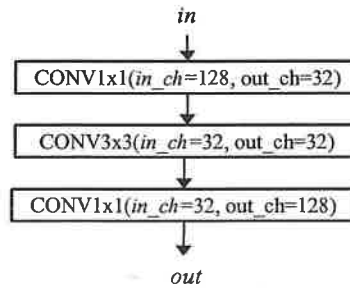


Figure 3.1: Plain Block

(4 marks)

- (b) Table 3.1 shows the configuration of a custom ResNet.

Table 3.1: A custom ResNet Architecture

Block	Layers	Shape of output Data format: (C, H, W)
	Settings for CONV: $f \times f$, out_channels	
conv1 (stem)	CONV7x7, 32, stride 2	?
	3x3 max pool, stride 2	
conv2	$\begin{bmatrix} \text{CONV3x3, 32} \\ \text{CONV3x3, 32} \end{bmatrix} \times 4$	?
conv3	$\begin{bmatrix} \text{CONV3x3, 64} \\ \text{CONV3x3, 64} \end{bmatrix} \times 4$	?
conv4	$\begin{bmatrix} \text{CONV3x3, 128} \\ \text{CONV3x3, 128} \end{bmatrix} \times 4$	?
fc1	Global average pool, FC (20 neurons, sigmoid)	?

- (i) What is the depth of the network? (2 marks)

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**Q3. (b) (Continued)**

- (ii) The shape of the input batch data is (3, 112, 112). Identify the shape of the output of the blocks in Table 3.1. (5 marks)
 - (iii) Justify if the network is designed to handle either *binary*, *multi-class* or *multi-label* classification. (2 marks)
 - (iv) Specify how to set `in_features` and `out_features` when you implement the fully connected layer in the `fc1` block with the PyTorch command `torch.nn.Linear(in_features, out_features)`. (2 marks)
 - (v) Suggest a modification to the network in Table 3.1 so that it is able to handle two different types of tasks, e.g., visual category classification and image quality classification, separately but concurrently. (2 marks)
- [Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE**SECTION B (Choose ONE question only)**

- Q4. (a) Low-Rank Adaptation (LoRA) is used to finetune a fully connected layer with parameter W with a shape of $d \times k$.
- (i) Specify the reason for performing PEFT. (3 marks)
 - (ii) Compare Additive vs Reparameterization PEFT methods. Which type of PEFT method does LoRA belong to? (3 marks)
 - (iii) Specify the main assumption behind LoRA. Then, write down the equation and explain how LoRA updates W by analyzing the dimensionality of the terms involved. (6 marks)
 - (iv) Given the shape of the network parameter W in a fully connected layer is $(20, 20)$ and the rank $r = 4$. Compare the number of parameters when finetuning with LoRA compared to full fine-tuning. (5 marks)
 - (v) Explain why LoRA scales better than full fine-tuning when adapting a LLM foundation model for multiple downstream tasks. (2 marks)
- (b) Masked Language Modeling (MLM) or Auto-regressive Language Modeling (ARLM) are two different self-supervised learning pretext tasks.
- (i) Which pretext task is better at capturing bidirectional relationships between words? Justify your answer. (3 marks)
 - (ii) Which pretext task is better at classification tasks such as named entity recognition? Justify your answer. (3 marks)
- [Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Q5. Answer the following questions on Multi-Head Self-Attention block (MHSA) in a transformer layer.

- (a) (i) Specify **TWO(2)** differences between Transformer and Recurrent Neural Networks (RNN). (4 marks)
- (ii) What are the roles of the query (**Q**), key (**K**), and value (**V**) vectors in self-attention? (3 marks)
- (iii) What is the reason of having multiple heads? (2 marks)
- (iv) Contrast how transformer encodes positional information when compared to RNN. (4 marks)
- (b) Given a batch data, the values of **V** and the scaled dot product are given below:

V		Softmax($\mathbf{QK}^T/\sqrt{d_k}$)		
1	-2	0.5	0.3	0.2
0	1	0.3	0.6	0.1
-3	5	0.2	0.1	0.7
(a)		(b)		

Figure 5.1: The values (a) and the softmax probabilities (b) of a batch data

- (i) What is the length of the token in the sequence? What is the number of features in each token? (2 marks)
- (ii) What is the probability that the 2nd token is relevant to the 3rd token? (2 marks)
- (iii) Compute the **attended value** for the 2nd token, $\hat{\mathbf{v}}^{<2>}$. (2 marks)
- (iv) What is the impact if we remove the denominator $\sqrt{d_k}$? (2 marks)
- (v) How can we modify the multi-head self-attention module to allow generative task? Hints: the formulation for MHSA is given in Figure 5.1(b). (4 marks)

[Total : 25 marks]

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Appendix

Logistic Regression

Forward propagation

$$z = w_1 x_1 + \dots + w_{F_x} x_{F_x} + b = \mathbf{x}^T \mathbf{w} + b$$

$$\mathbf{z} = \mathbf{X}\mathbf{w} + b$$

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{z}) = \frac{1}{1+e^{-\mathbf{z}}}$$

Gradient Descent

$$\mathbf{w} := \mathbf{w} - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial \mathbf{w}}$$

$$b := b - \alpha \frac{\partial J(\mathbf{w}, b)}{\partial b}$$

$$\frac{\partial J}{\partial \hat{y}} = \frac{1}{M} \left(\frac{1-y}{(1-\hat{y})} - \frac{y}{\hat{y}} \right)$$

$$\frac{\partial J}{\partial \mathbf{z}} = \frac{\partial J}{\partial \hat{y}} * (\hat{y} * (1 - \hat{y}))$$

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{X}^T \frac{\partial J}{\partial \mathbf{z}}$$

$$\frac{\partial J}{\partial b} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{z}} \right)^T$$

Standard Neural Network

Forward propagation

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]} \mathbf{W}^{[l]} + \mathbf{b}^{[l]}$$

$$\mathbf{A}^{[l]} = g(\mathbf{Z}^{[l]})$$

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \text{ (softmax)}$$

Gradient Descent

$$\frac{\partial J}{\partial \hat{\mathbf{y}}} = \frac{1}{C \times B} \left(\frac{1-Y}{1-\hat{Y}} - \frac{Y}{\hat{Y}} \right)$$

$$\frac{\partial J}{\partial \mathbf{Z}^{[l]}} = \begin{cases} \frac{\partial J}{\partial \mathbf{A}^{[l]}} * \mathbf{A}^{[l]} * (1 - \mathbf{A}^{[l]}) & \text{for sigmoid activation} \\ \frac{\partial J}{\partial \mathbf{A}^{[l]}} * (\mathbf{Z}^{[l]} > 0) & \text{for relu activation} \end{cases}$$

$$\frac{\partial J}{\partial \mathbf{W}^{[l]}} = \mathbf{A}^{[l-1]T} \frac{\partial J}{\partial \mathbf{Z}^{[l]}}$$

$$\frac{\partial J}{\partial \mathbf{b}^{[l]}} = \left(\mathbf{1}^T \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \right)^T$$

$$\frac{\partial J}{\partial \mathbf{A}^{[l-1]}} = \frac{\partial J}{\partial \mathbf{Z}^{[l]}} \mathbf{W}^{[l]T}$$

Activation Functions

Sigmoid

$$g(z) = \frac{1}{1+e^{-z}}, \quad \frac{dg}{dz} = g(z) \cdot (1 - g(z))$$

Tanh

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \frac{dg}{dz} = 1 - g(z)^2$$

ReLU

$$g(z) = \max(0, z), \quad \frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0 & \text{if } z < 0 \end{cases}$$

Leaky ReLU

$$g(z) = \max(0.01z, z), \quad \frac{dg}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \\ 0.01 & \text{if } z < 0 \end{cases}$$

Cost Function

Binary cross entropy

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i=1}^B -y^{(i)} \ln \hat{y}^{(i)} - (1 - y^{(i)}) \ln (1 - \hat{y}^{(i)}) \\ = \text{mean}(-\mathbf{y} * \ln \hat{\mathbf{y}} - (1 - \mathbf{y}) * \ln (1 - \hat{\mathbf{y}}))$$

Binary cross entropy (multi-label)

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{C \times B} \sum_{c=1}^C \sum_{i=1}^B -y_c^{(i)} \ln \hat{y}_c^{(i)} - (1 - y_c^{(i)}) \ln (1 - \hat{y}_c^{(i)}) \\ = \text{mean}(-\mathbf{Y} * \ln \hat{\mathbf{Y}} - (1 - \mathbf{Y}) * \ln (1 - \hat{\mathbf{Y}}))$$

Negative Log Likelihood

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B} \sum_{i=1}^B -\ln(p_c^{(i)})$$

Convolutional Neural Network

$$\mathbf{A}^{[l]}(i, j) = \mathbf{A}_{loc}^{[l-1]}(i^*, j^*) \cdot \mathbf{W}^{[l]} + b^{[l]}$$

$$o = \left\lfloor \frac{i+2p-f}{s} + 1 \right\rfloor$$

$$\# \text{params in a filter} = c_f \times h_f \times w_f + 1$$

$$\# \text{params in a CONV layer} = \# \text{filters} \times \# \text{params in a filter}$$

$$\# \text{computations} = \# \text{output activations} \times \# \text{computations per activation} \\ = \text{output volume size} \times \# \text{params in a filter}$$

Efficient Net

$$\text{depth } d = \alpha^\phi, \text{ width } w = \beta^\phi, \text{ resolution } r = \gamma^\phi$$

$$\text{subject to: } \alpha, \beta^2, \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

RNN and LSTM

RNN

$$\mathbf{H}^{<t>} = g(\mathbf{H}^{<t-1>} \mathbf{W}_{hh} + \mathbf{X}^{<t>} \mathbf{W}_{hx} + \mathbf{b}^T)$$

$$\mathbf{H}^{<t>} = g([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_h + \mathbf{b}^T)$$

LSTM

$$\mathbf{F}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_f + \mathbf{b}_f^T)$$

$$\mathbf{I}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_i + \mathbf{b}_i^T)$$

$$\mathbf{O}^{<t>} = \sigma([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_o + \mathbf{b}_o^T)$$

$$\tilde{\mathbf{C}}^{<t>} = \tanh([\mathbf{H}^{<t-1>} \quad \mathbf{X}^{<t>}] \mathbf{W}_c + \mathbf{b}_c^T)$$

$$\mathbf{C}^{<t>} = \mathbf{I}^{<t>} * \tilde{\mathbf{C}}^{<t>} + \mathbf{F}^{<t>} * \mathbf{C}^{<t-1>}$$

$$\mathbf{H}^{<t>} = \mathbf{O}^{<t>} * \tanh(\mathbf{C}^{<t>})$$

Transformers

$$\hat{\mathbf{V}} = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right) \mathbf{V}$$

$$\hat{\mathbf{V}}_m = \text{softmax}\left(\frac{\mathbf{QK}^T + \mathbf{M}}{\sqrt{d_k}}\right) \mathbf{V}$$

UCCD3074 DEEP LEARNING FOR DATA SCIENCE

Appendix

Optimizers

Momentum

$$\mathbf{v}^{<t>} := \beta \mathbf{v}^{<t-1>} + (1 - \beta) \frac{\partial J}{\partial \mathbf{w}}^{<t>}$$

$$\mathbf{v}^{<t>} := \frac{1}{1 - \beta^t} \mathbf{v}^{<t>}$$

$$\mathbf{w}^{<t>} := \mathbf{w}^{<t-1>} - \alpha \mathbf{v}^{<t>}$$

$$\text{weight}^{<t-n>} = (1 - \beta) \beta^n$$

RMSProp

$$\mathbf{s}^{<t>} := \beta \mathbf{s}^{<t>} + (1 - \beta) \left(\frac{\partial J}{\partial \mathbf{w}}^{<t>} \right)^2$$

$$\mathbf{w}^{<t>} := \mathbf{w}^{<t-1>} - \left(\frac{\alpha}{\sqrt{\mathbf{s}^{<t>} + \epsilon}} \right) \frac{\partial J}{\partial \mathbf{w}}^{<t>}$$

Adam

$$\mathbf{v}^{<t>} := \beta_1 \mathbf{v}^{<t-1>} + (1 - \beta_1) \frac{\partial J}{\partial \mathbf{w}}^{<t>}$$

$$\tilde{\mathbf{v}}^{<t>} = \frac{\mathbf{v}^{<t>}}{(1 - \beta_1^t)}$$

$$\mathbf{s}^{<t>} := \beta_2 \mathbf{s}^{<t-1>} + (1 - \beta_2) \left(\frac{\partial J}{\partial \mathbf{w}}^{<t>} \right)^2$$

$$\tilde{\mathbf{s}}^{<t>} = \frac{\mathbf{s}^{<t>}}{(1 - \beta_2^t)}$$

$$\mathbf{w}^{<t>} = \mathbf{w}^{<t-1>} - \left(\frac{\alpha}{\sqrt{\tilde{\mathbf{s}}^{<t>} + \epsilon}} \right) \tilde{\mathbf{v}}^{<t>}$$

Learning Rate Decay

$$\alpha^{<t>} = \frac{1}{1 + rt} \alpha^{<0>}$$

$$\alpha^{<t>} = \gamma^{t/T} \alpha^{<0>}$$

$$\alpha^{<t>} = e^{-k \times t} \alpha^{<0>}$$

Initialization

Xavier Initialization

$$\text{var}(\mathbf{W}^{[l]}) = \frac{2}{(F^{[l-1]} + F^{[l]})}$$

Kaiming Initialization

$$\text{var}(\mathbf{W}^{[l]}) = \frac{2}{F^{[l-1]}}$$

Normalization

BatchNorm

$$\hat{\mathbf{Z}}[:, f] = \frac{\mathbf{Z}[:, f] - \mu_f}{\sqrt{v_f + \epsilon}}$$

$$\hat{\mathbf{Z}}[:, c, :, :] = \frac{\mathbf{Z}[:, c, :, :] - \mu_c}{\sqrt{v_c + \epsilon}}$$

$$\tilde{\mathbf{Z}}[:, c, :, :] = \gamma_c \hat{\mathbf{Z}}[:, c, :, :] + \beta_c$$

$$\mu_c^{(t)} = \text{mean}(\mathbf{Z}^{(t)}[:, c, :, :]) \quad (\text{training mode})$$

$$v_c^{(t)} = \text{variance}(\mathbf{Z}^{(t)}[:, c, :, :]) \quad (\text{training mode})$$

$$\tilde{\mu}_c = (1 - m) \tilde{\mu}_c + m \mu_c^{(t)} \quad (\text{eval mode})$$

$$\tilde{v}_c = (1 - m) \tilde{v}_c + m v_c^{(t)} \quad (\text{eval mode})$$

LayerNorm

$$\hat{\mathbf{Z}}[i, t, :] = \frac{(\mathbf{Z}[i, t, :] - \mu^{(i)<t>})}{\sqrt{v^{(i)<t>} + \epsilon}}$$

$$\tilde{\mathbf{Z}}[i, t, :] = \gamma^{(i)<t>} \hat{\mathbf{Z}}[i, t, :] + \beta^{(i)<t>}$$

$$\mu^{(i)<t>} = \text{mean}(\mathbf{Z}[i, t, :]) \quad (\text{training and eval mode})$$

$$v^{(i)<t>} = \text{variance}(\mathbf{Z}[i, t, :]) \quad (\text{training and eval mode})$$

